Name: **CSE 450, Fall 2026 – Quiz 2B (Parsing)**

Each question is worth 1 point, for 10 points total. Choose one answer per question. Questions 9 and 10 concern your submitted Project 1. Those answers vary by student: describe the code you submitted, including any incomplete behavior, rather than the implementation you think would be best.

1. Which grammar gives '@' higher precedence than '$', with BOTH operators LEFT associative? Use E as the start symbol and NUM as a numeric-literal token.

Hint: Check both the precedence levels and the direction of recursion.

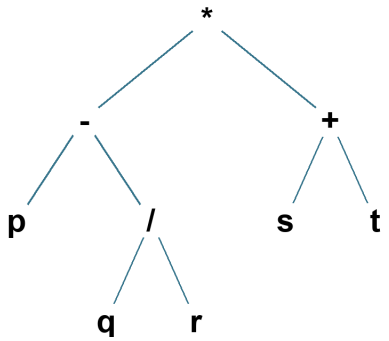
A.
```
EXPR2 -> EXPR2 '@' EXPR1 | EXPR1
EXPR1 -> EXPR1 '$' TERM | TERM
TERM  -> NUM | '(' EXPR2 ')'
```

B.
```
EXPR2 -> EXPR1 '@' EXPR2 | EXPR1
EXPR1 -> TERM '$' EXPR1 | TERM
TERM  -> NUM | '(' EXPR2 ')'
```

C.
```
EXPR2 -> EXPR2 '$' EXPR1 | EXPR1
EXPR1 -> EXPR1 '@' TERM | TERM
TERM  -> NUM | '(' EXPR2 ')'
```

D.
```
EXPR2 -> EXPR1 '$' EXPR2 | EXPR1
EXPR1 -> TERM '@' EXPR1 | TERM
TERM  -> NUM | '(' EXPR2 ')'
```

2. Using standard precedence and associativity, which expression produces this AST?



A. `p - q / r * (s + t)`

B. `(p - q) / r * (s + t)`

C. `(p - q / r) * (s + t)`

D. `(p - q / r) * s + t`

3. Given the expression:

```
result = (m + n) * p - q / (r - s)
```

Which calculation is completed FIRST when the AST is executed? Assume standard precedence and associativity. Evaluate each binary operator's left subtree completely before its right subtree.

Hint: An operator executes only after its operands have been evaluated.

A. `m + n`

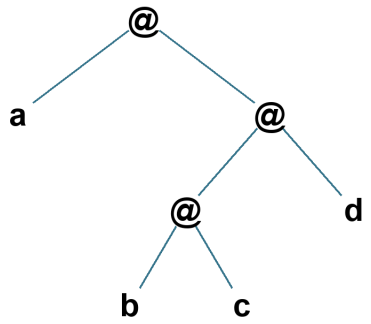
B. `p - q`

C. `q / (r - s)`

D. `r - s`

E. The assignment to result

4. The binary operator @ is RIGHT associative. Which expression produces this AST?



A. `a @ b @ c @ d`

B. `(a @ b) @ c @ d`

C. `a @ (b @ c) @ d`

D. `(a @ b @ c) @ d`

E. No expression can produce this tree with right associativity.

5. An expression AST contains only operation and operand nodes. Compare the ASTs for

```
((u - v)) * w
```

and

```
(u - v) * w
```

Which statement is correct?

A. The ASTs have identical nodes and connections.

B. The first AST must contain one extra subtraction node to represent the extra grouping.

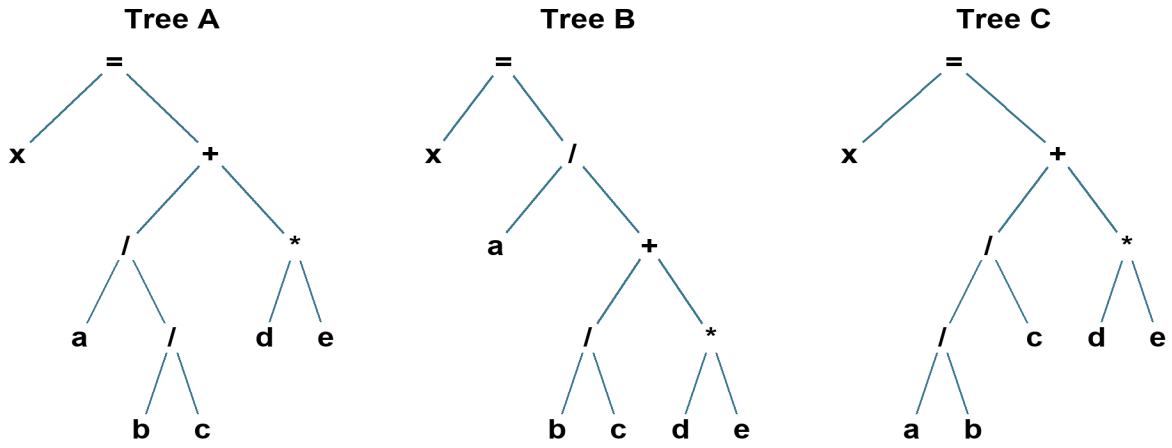
C. The first AST must contain an extra parenthesis node that reverses the order of evaluation.

D. The ASTs have the same nodes, but in a different configuration such that the first must execute multiplication before subtraction.

6. Given the expression:

```
x = a / b / c + d * e;
```

Which AST is produced using standard precedence and associativity, before any optimization?



Hint: With a = 24, b = 6, c = 2, d = 3, and e = 4, the expression assigns 14 to x. Check the tree structure as well as the result.

A. Tree A only

B. Tree B only

C. Tree C only

D. More than one of these trees

E. None of these trees

7. A WHILE-loop AST node has two children: a condition and a body. In the interpreter you find the following code, but it's not always producing the correct result:

```
ASTNode & condition = node.children[0];
ASTNode & body = node.children[1];
int64_t value = Interpret(condition);
while (value != 0) {
  Interpret(body);
}
```

Which change do you need to make to repair this implementation?

A. Run the body *before* testing the condition, then test the condition after each iteration.

B. Ignore value and stop when `Interpret(body)` returns zero.

C. Run `Interpret(condition)` *twice* before entering the loop, then use the second result.

D. Evaluate `Interpret(condition)` before every iteration. Stop looping on zero.

E. This code works correctly as-is.

8. Consider this ambiguous grammar for the binary operator **:

```
S -> S '**' S | '(' S ')' | NUM
```

Which replacement is unambiguous, makes ** RIGHT associative, and preserves the same language, including parenthesized expressions? In each choice, S is the start symbol and NUM is a numeric-literal token.

A.
```
S -> T '**' S | T
T -> NUM | '(' S ')'
```

B.
```
S -> S '**' T | T
T -> NUM | '(' S ')'
```

C.
```
S -> T '**' T | T
T -> NUM | '(' S ')'
```

D.
```
S -> T '**' S | T
T -> NUM | '(' T ')'
```

9. In your submitted Project 1, how did you detect the end of a command's line to know that you should stop collecting arguments? (you may ignore end-of-file handling)

A. I compared token **line numbers** (i.e., `line_id`), and ignored newline tokens / characters.

B. I checked for **newline tokens** or newline characters in token lexemes, and ignored build-in line numbers for knowing when to stop collecting arguments.

C. I used **both** token-line-number comparisons and checks for newline tokens / characters to stop argument collection.

D. I split raw input into lines **before tokenizing**.

E. This part of the interpreter was not implemented or I used a completely different approach.

10. When executing LOAD name with name as a variable, how did your submitted Project 1 FIRST check whether that variable had a stored value? Include checks inside helper functions.

If the lookup uses more than one container operation, choose the initial existence check, not a later retrieval operation.

A. It called find(name) and compared the returned iterator with end().

B. It called count(name) or contains(name) to test whether the name was present.

C. It called at(name) and caught an out_of_range exception when the name was absent.

D. It read the map using operator[] without first checking whether the name existed.

E. It used a different approach, or loading a variable was not implemented.